

ПРОМЫШЛЕННОЕ МАШИННОЕ ОБУЧЕНИЕ VIII-IX

Муратов Симар

Осень
2022

Содержание

1. База данных;
2. SQL СУБД;
3. NoSQL СУБД;
4. Data Warehouse;
5. Data Lake;
6. Data Mart;
7. ClickHouse;
8. HDFS;
9. GreenPlum;
10. Основы инжиниринга данных.

База данных: определение

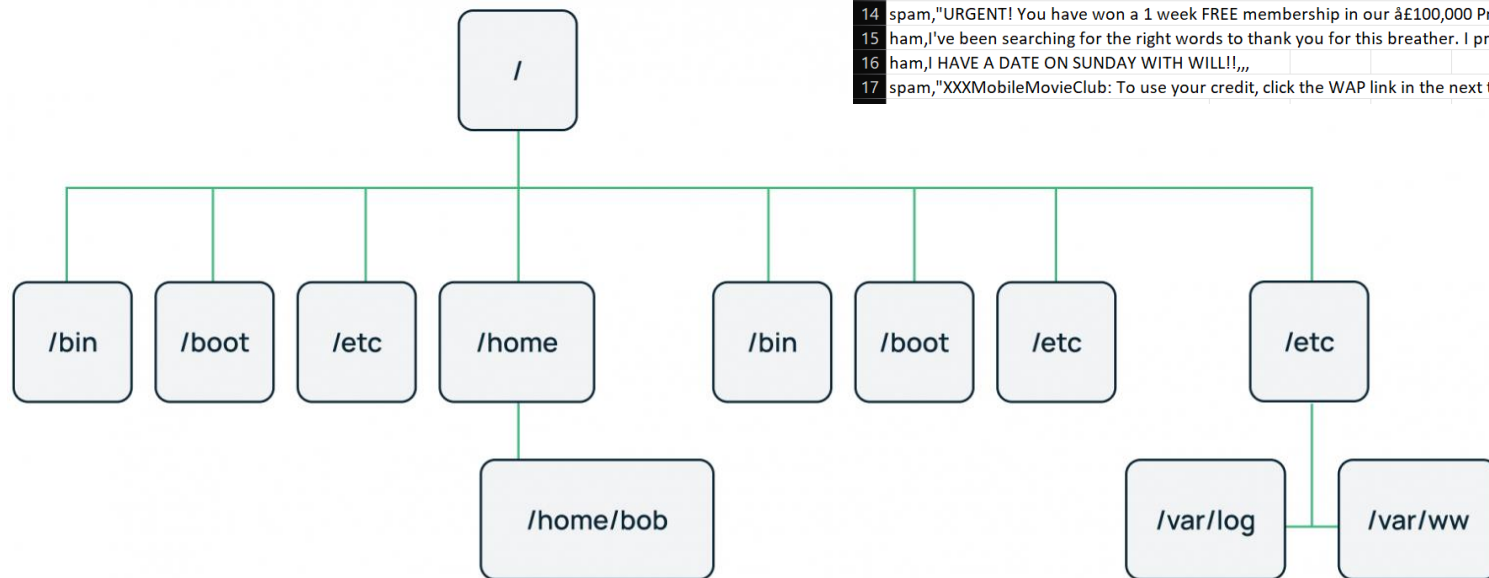
База данных — это набор сведений об объектах, структурированный определенным образом. Обычно базы данных управляются специальным ПО, или системами управления базами данных (СУБД).

В зависимости от вида логическая структура базы данных может иметь различное описание. Это различие влияет на то, какая именно БД используется в разработке конкретного продукта или технологии.

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
syslog:x:102:106::/home/syslog:/usr/sbin/nologin
```

База данных: самое начало

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	A																		
2	ham,"Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat..."																		
3	ham,Ok lar... Joking wif u oni.....																		
4	spam,Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's,,																		
5	ham,U dun say so early hor... U c already then say.....																		
6	ham,"Nah I don't think he goes to usf, he lives around here though"																		
7	spam,"FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, â€1.50 to rcv" ,,,																		
8	ham,Even my brother is not like to speak with me. They treat me like aids patent.,,,																		
9	ham,As per your request 'Melle Melle (Oru Minnaminunginte Nurun Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune.,,,																		
10	spam,WINNER!! As a valued network customer you have been selected to receive a â€900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.,,,																		
11	spam,Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co FREE on 08002986030.,,,																		
12	ham,"I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today." ,,,																		
13	spam,"SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply HL 4 info" ,,,																		
14	spam,"URGENT! You have won a 1 week FREE membership in our â€100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18" ,,,																		
15	ham,I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. You have been wonderful and a blessing at all times.,,,																		
16	ham,I HAVE A DATE ON SUNDAY WITH WILL!!,,																		
17	spam,"XXXMobileMovieClub: To use your credit, click the WAP link in the next txt message or click here>> http://wap.xxxmobilemovieclub.com?n=QJGIGHJGCB" ,,,																		



SQL СУБД: определение

SQL СУБД или реляционная база данных – это набор высокоструктурированных таблиц, в которых каждая строка отражает объект данных, а каждый столбец определяет конкретное информационное поле.

Реляционные базы данных строятся с использованием языка структурированных запросов (SQL) для создания, хранения, обновления и извлечения данных.



SQL СУБД: структура данных

Сервер баз данных SQL хранит и организует данные в **таблицах**. В РСУБД таблицы являются фундаментальными объектами базы данных, логически спроектированными для сбора данных в формате строк и столбцов.

В то время как строки отражают сущности, столбцы определяют атрибуты каждой сущности.



SQL СУБД: создание таблиц

Каждый **столбец** содержит определенную атрибутивную информацию, а свойства столбца определяют тип данных (например, числовые или текстовые данные) и диапазон, который он может принимать.

Каждая таблица имеет **первичный ключ** для уникальной идентификации сущности.

```
CREATE TABLE accounts (  
    user_id serial PRIMARY KEY,  
    username VARCHAR ( 50 ) UNIQUE NOT NULL,  
    password VARCHAR ( 50 ) NOT NULL,  
    email VARCHAR ( 255 ) UNIQUE NOT NULL,  
    created_on TIMESTAMP NOT NULL,  
    last_login TIMESTAMP  
);
```

SQL СУБД: запросы

Пользователи базы данных могут добавлять данные в каждую **строку** и выполнять SQL-запросы для получения данных.

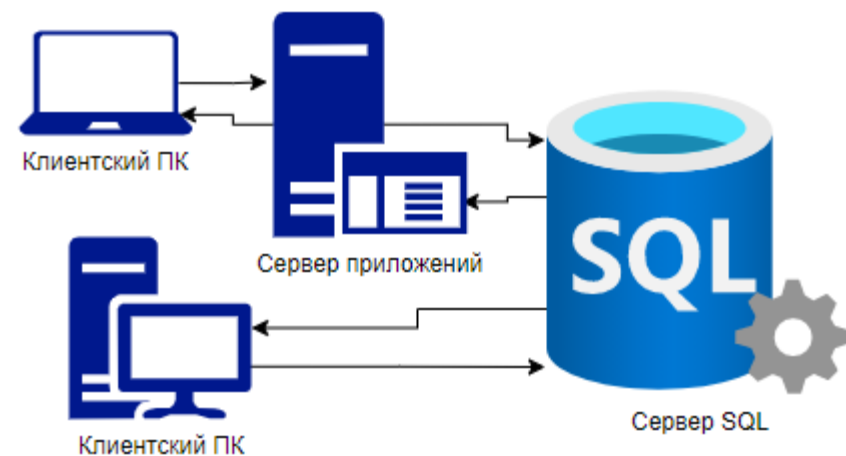
Для первичного ключа каждая строка содержит уникальное значение, что также помогает преодолеть проблемы дублирования данных.

```
SELECT
    film_id,
    title,
    rental_rate
INTO TABLE film_r
FROM
    film
WHERE
    rating = 'R'
AND rental_duration = 5
ORDER BY
    title;
```

SQL СУБД: гибкость

Поскольку стандартным языком программирования является SQL, реляционные базы данных используют DDL для изменения схемы в режиме реального времени.

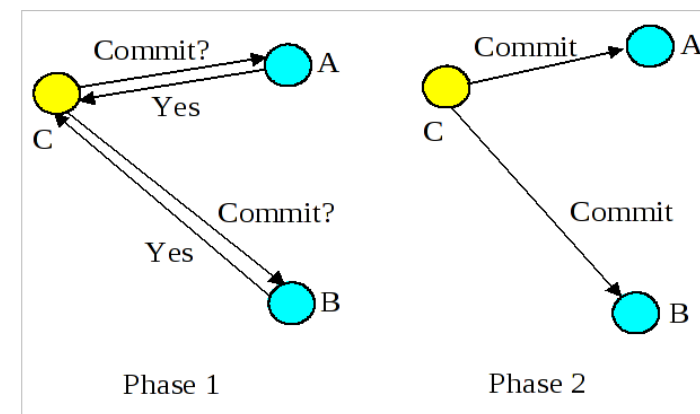
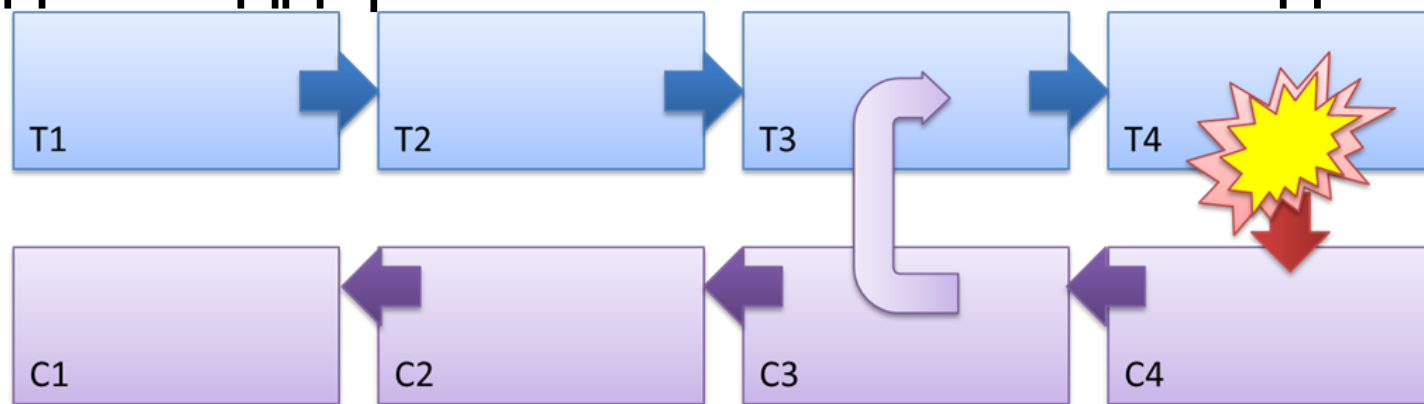
Это позволяет пользователям баз данных добавлять новые таблицы и столбцы, переименовывать отношения и вносить различные другие изменения в режиме реального времени **без остановки** операций с базой данных.



SQL СУБД: согласованность данных

Базы данных SQL эффективно поддерживают согласованность данных между приложениями и экземплярами сервера баз данных SQL. Другие типы баз данных с трудом поддерживают согласованность данных в **реальном времени** при больших объемах данных.

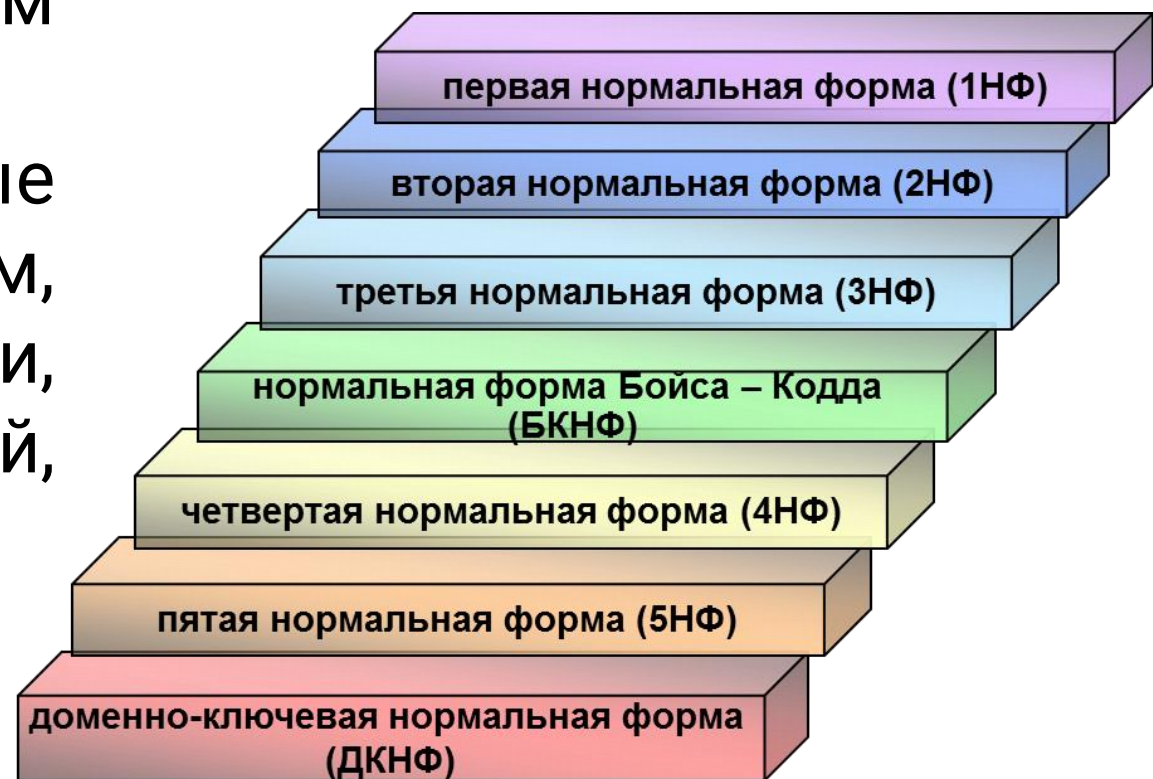
Критически важные приложения, обрабатывающие важные бизнес-транзакции, полагаются на реляционные базы данных для поддержания согласованности данных.



SQL СУБД: минимальная избыточность

SQL СУБД уменьшают избыточность данных путем реализации **нормализации**.

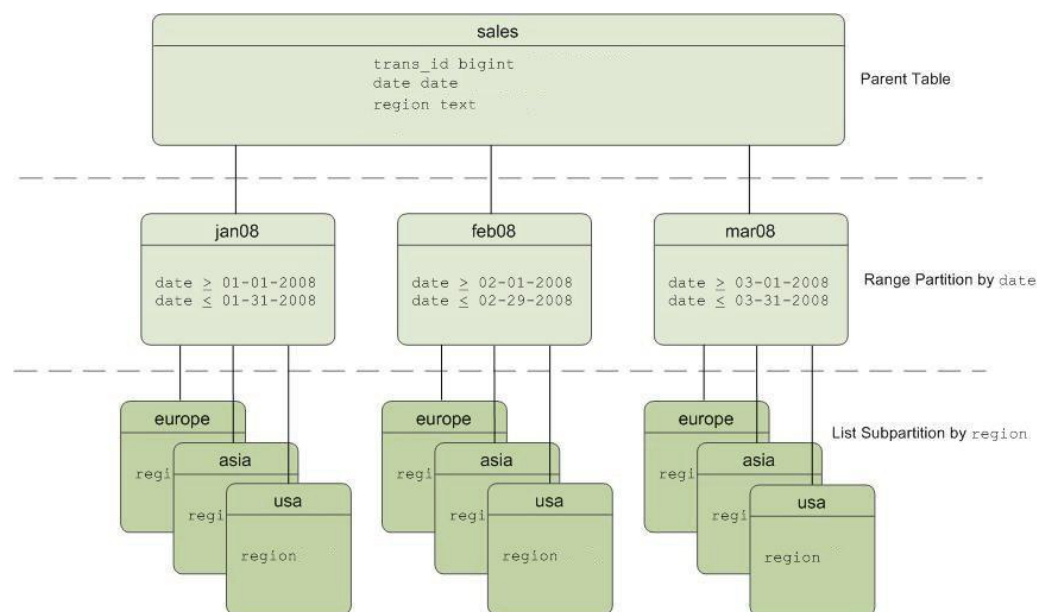
При нормализации данные организуются таким образом, чтобы устранить аномалии, связанные со вставкой, обновлением и удалением данных.



SQL СУБД: оптимизация производительности

Реляционные базы данных, обладающие целым рядом дополнительных функций, обеспечивают простоту и скорость выполнения операций с базами данных.

Минимальное использование памяти, снижение затрат на хранение данных и высокая скорость процессора помогают **ускорить работу** базы данных для всех приложений.

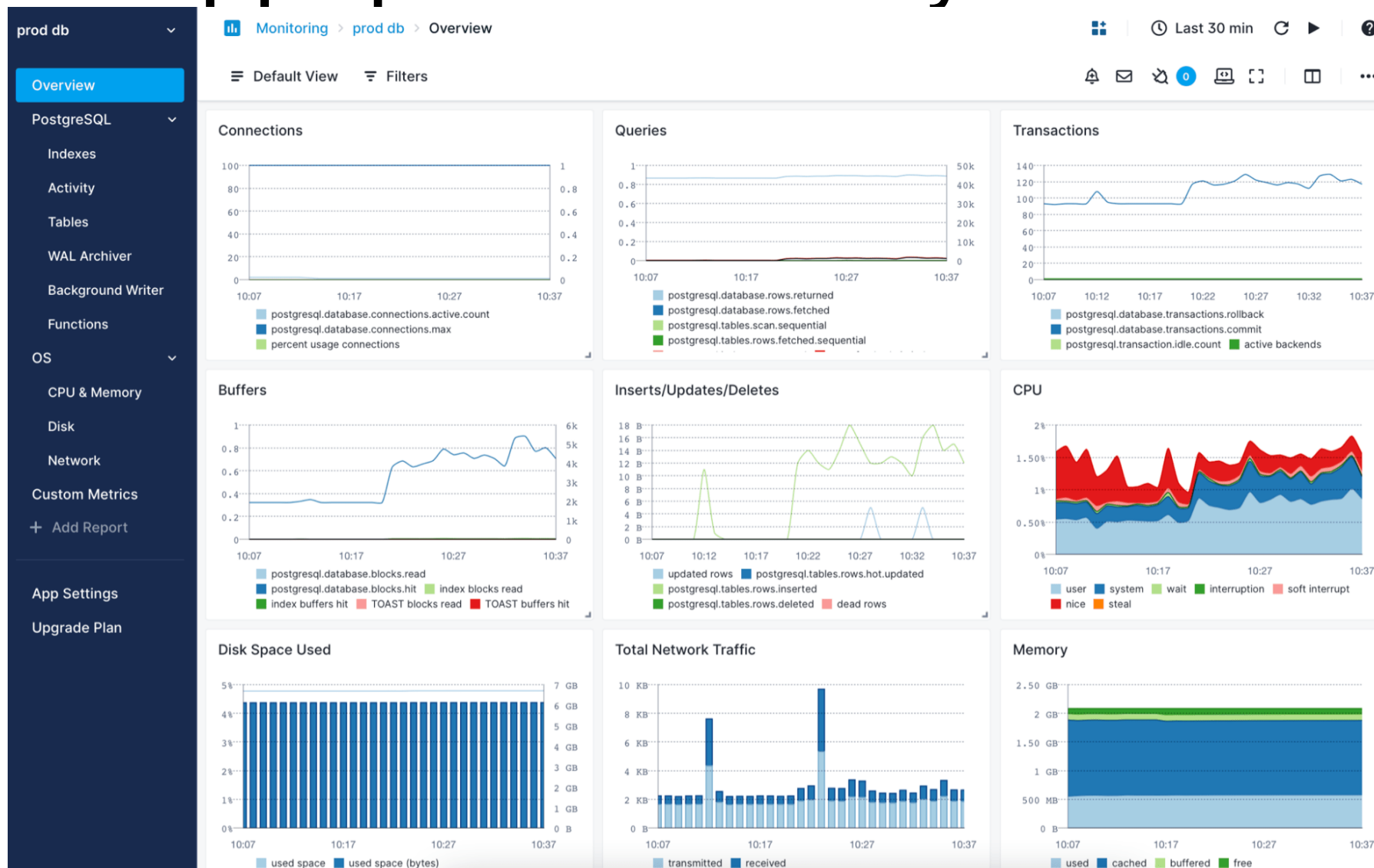


SQL СУБД: простота обслуживания

Встроенные средства автоматизации в системах реляционных баз данных помогают упростить ремонт, контроль и обслуживание баз данных SQL. Это облегчает администраторам баз данных и техническим специалистам активное обслуживание и обновление базы данных.

Кроме того, использование внешних инструментов для мониторинга баз данных SQL может дать возможность управлять базой данных в индивидуальном порядке для каждой организации, использующей базы данных.

SQL СУБД: простота обслуживания



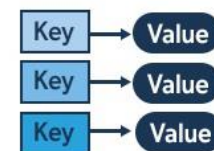
NoSQL СУБД: определение

Базы данных **NoSQL** (они же "не только SQL") являются нетабличными базами данных и хранят данные иначе, чем реляционные таблицы. Базы данных NoSQL бывают разных типов в зависимости от модели данных.

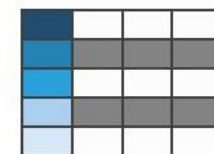
Основными типами являются документ, ключ-значение, широкий столбец и граф. Они обеспечивают гибкие схемы и легко масштабируются при больших объемах данных и высокой пользовательской нагрузке.

NoSQL

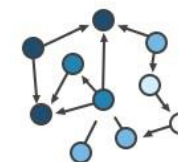
Key-Value



Column-Family



Graph



Document



NoSQL СУБД: не используется SQL

NoSQL СУБД **не** используют ANSI SQL DML, так как многие базы пытаются использовать query languages похожие на общеизвестный любимый синтаксис, но полностью его реализовать не удалось никому и вряд ли удастся.



NoSQL СУБД: неструктурированные

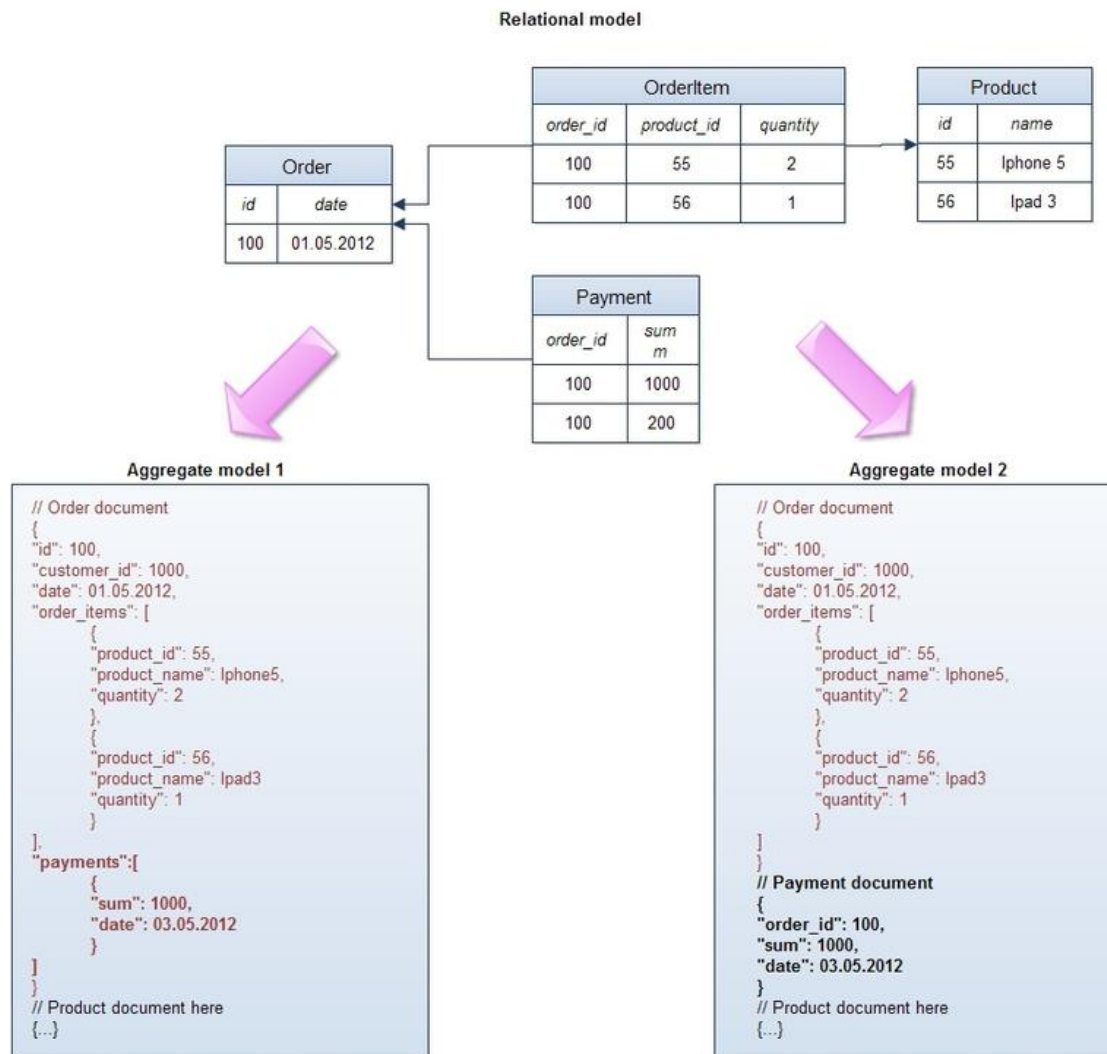
Смысл таков, что в NoSQL базах в отличие от реляционных структура данных не регламентирована (или слабо типизированна, если проводить аналогии с языками программирования) — в отдельной строке или документе можно добавить произвольное поле без предварительного декларативного изменения структуры всей таблицы.

Таким образом, если появляется необходимость поменять модель данных, то единственное достаточное действие — отразить изменение в коде приложения.

NoSQL СУБД: агрегаты

В отличие от реляционной модели, которая сохраняет логическую бизнес-сущность приложения в различные физические таблицы в целях нормализации, NoSQL хранилища оперируют с этими сущностями как с целостными объектами.

NoSQL СУБД: агрегаты



NoSQL СУБД: агрегаты

Нормализация данных	Данные в виде агрегатов
• Целостность информации при обновлении (меняем запись в одной таблице, а не в нескольких) • Ориентированность на широкий спектр запросов к данным	• Оптимизация только под определенный вид запросов • Сложности при обновлении денормализованных данных
• Неэффективна в распределенной среде • Низкая скорость чтения при использовании объединений (joins) • Несоответствие объектной модели приложения физической структуре данных (impedance mismatch, решается с помощью Hibernate etc.)	• Лучший способ добиться большой скорости на чтение в распределенной среде • Возможность хранить физически объекты в том виде, в каком с ними работает приложение (легче кодировать и меньше ошибок при преобразовании) • Родная (native) поддержка атомарности на уровне записей

NoSQL СУБД: слабые ACID свойства

Долгое время консистентность (consistency) данных была “священной коровой” для архитекторов и разработчиков. Все реляционные базы обеспечивали тот или иной уровень изоляции — либо за счет блокировок при изменении и блокирующего чтения, либо за счет undo-логов.

С приходом огромных массивов информации и распределенных систем стало ясно, что обеспечить для них транзакционность набора операций с одной стороны и получить высокую доступность и быстрое время отклика с другой — невозможно.

NoSQL СУБД: слабые ACID свойства

ACID – требования

ACID – требования гарантируют правильность и надежность работы системы

Atomic
(атомарность)

Транзакция не может выполняться частично, либо все, либо ничего

Consistency
(согласованность)

После выполнения транзакции все данные должны находиться в согласованном состоянии

Isolation
(изолированность)

Транзакция должна быть автономной и воздействовать на другие транзакции или зависеть от них

Durability
(устойчивость)

После завершения транзакции, внесенные изменения останутся неизменными

ACID - фундаментальные свойства систем обработки транзакций

NoSQL СУБД: слабые ACID свойства



NoSQL СУБД: распределенные системы

С лавинообразным ростом информации в мире и необходимости ее обрабатывать за разумное время встала проблема вертикальной масштабируемости — рост скорости процессора остановился на 3.5 ГГц, скорость чтения с диска также растет тихими темпами, плюс цена мощного сервера всегда больше суммарной цены нескольких простых серверов.

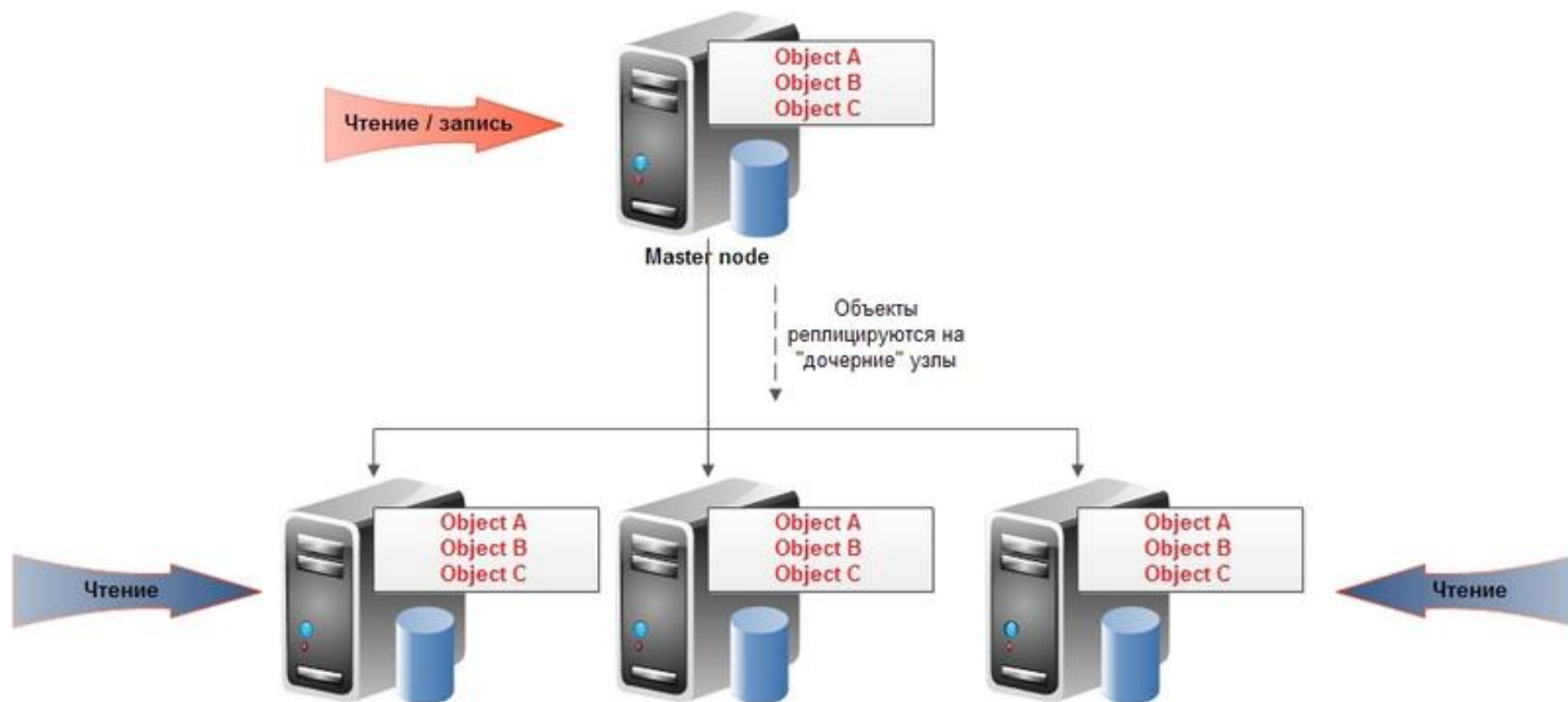
В этой ситуации обычные реляционные базы, даже кластеризованные на массиве дисков, не способны решить проблему скорости, масштабируемости и пропускной способности.

NoSQL СУБД: распределенные системы

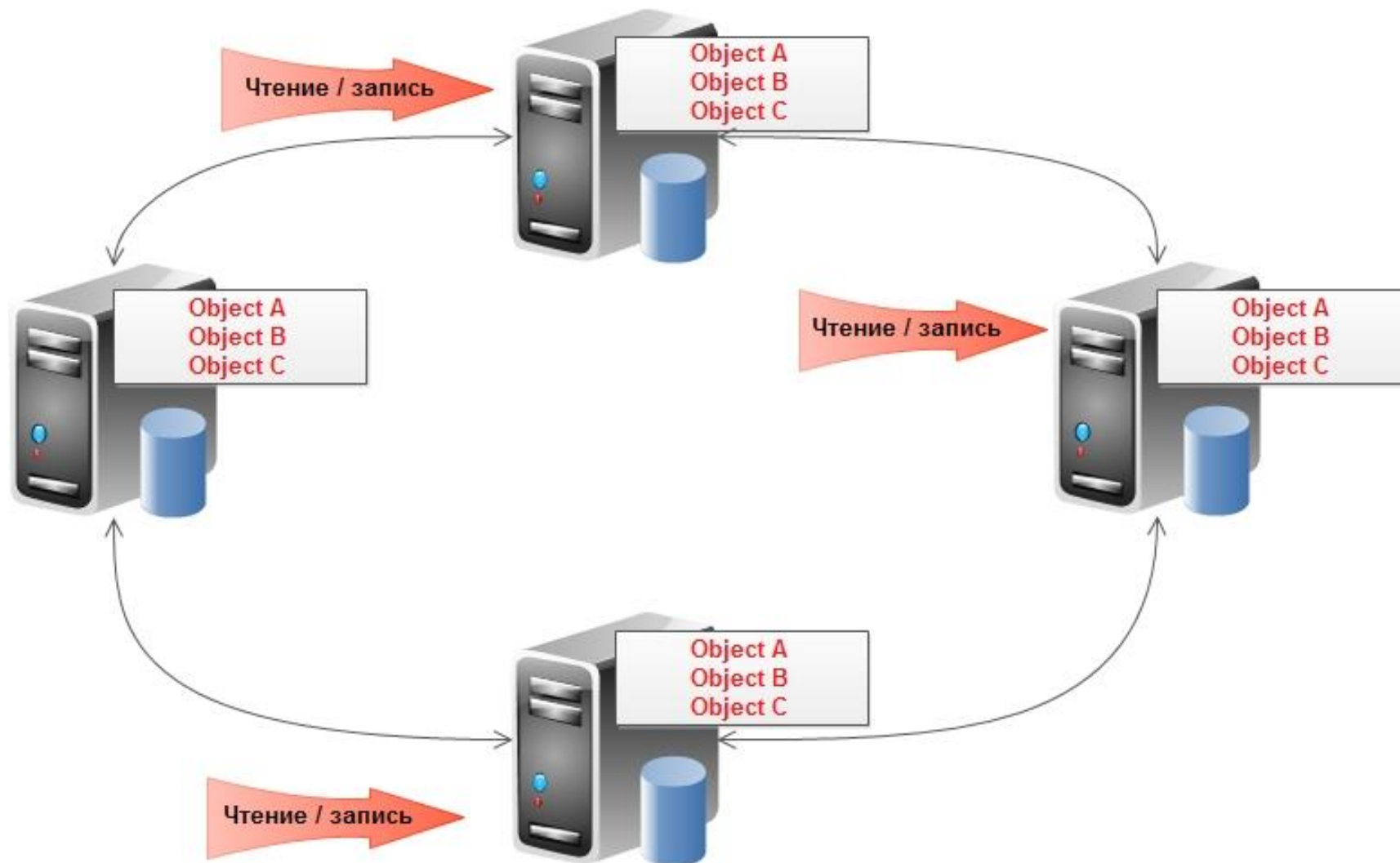
Единственный выход из ситуации — горизонтальное масштабирование, когда несколько независимых серверов соединяются быстрой сетью и каждый владеет/обрабатывает только часть данных и/или только часть запросов на чтение-обновление.

Репликация — копирование данных на другие узлы при обновлении. Позволяет как добиться большей масштабируемости, так и повысить доступность и сохранность данных. Принято подразделять на два вида.

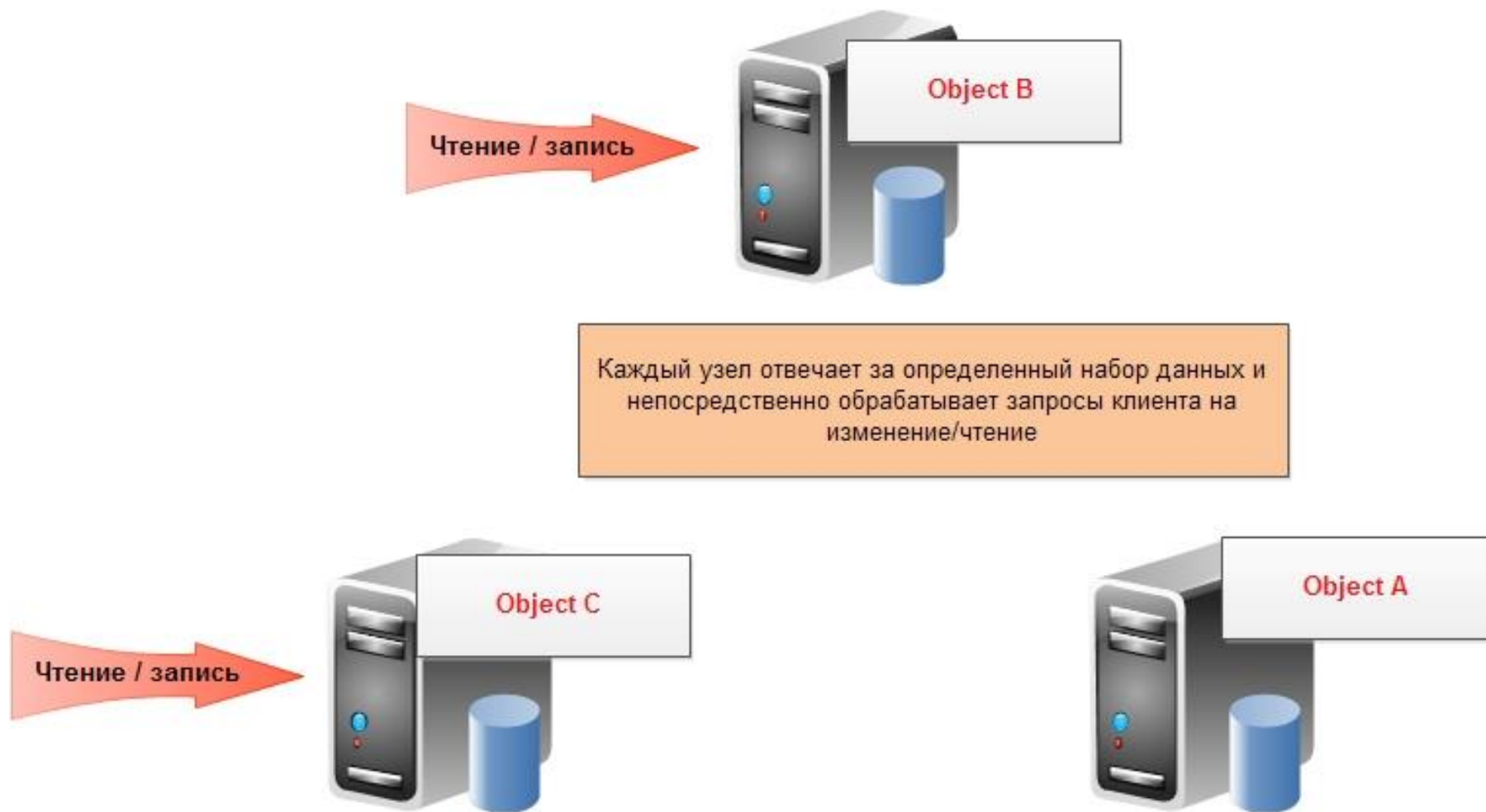
NoSQL СУБД: master-slave



NoSQL СУБД: peer-to-peer



NoSQL СУБД: шардинг



NoSQL vs SQL



VS



Прост в применении

Принцип ACID

Структура лучше

Трудоемкий дизайн

Вертикальное
масштабирование

Фиксированная схема

Быстрое развитие

Принцип BASE

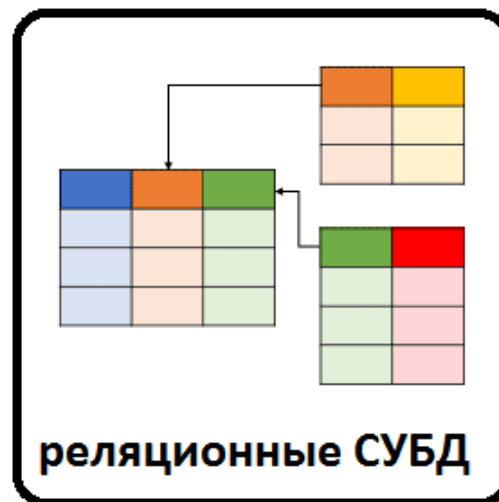
Работает быстрее

Нет инвестиций в дизайн

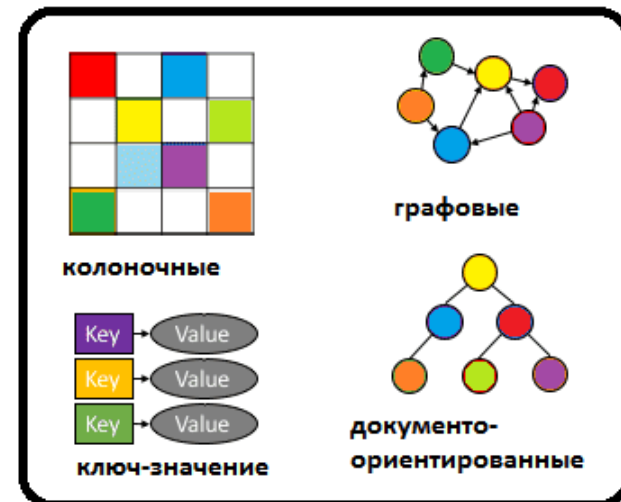
Горизонтальное
масштабирование

Динамическая Схема

SQL



NoSQL



Data Warehouse: определение

Data Warehouse — это единое корпоративное хранилище архивных данных из разных источников (систем, департаментов и прочее). Цель Data Warehouse — обеспечить пользователя (компанию и ее ключевых лиц) возможностью принимать верные решения в ключе управления бизнесом на основе целостной информационной картины.

Data Warehouse: особенности

1) Тип и источник данных

Обычные БД хранят данные от конкретных информационных систем компании. Например, в базе данных HR-отдела мы увидим информацию о сотрудниках, а вот данных о поставках там не будет. DWH строится по другому принципу: такое корпоративное хранилище консолидирует в себе информацию от всех департаментов компании — от статистики продаж до сведений о сотрудниках.

Data Warehouse: особенности

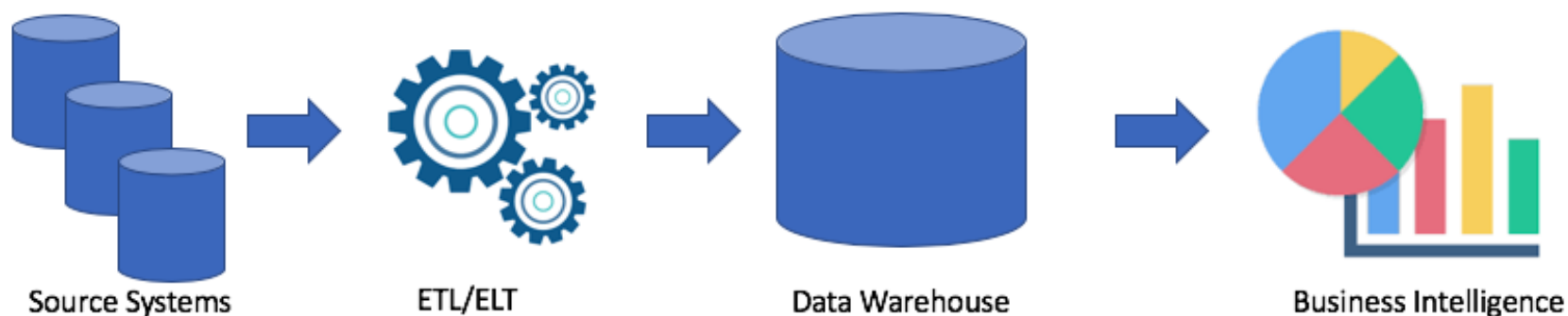
2) Объем данных

Обычные базы используются для хранения только актуальной информации — в ней не имеет смысла хранить данные за несколько лет работы предприятия. В Data Warehouse, наоборот, стекаются исторические данные и архивные сведения. Например, заглянув в DWH, можно получить информацию о всех сделках за последние несколько лет.

Data Warehouse: особенности

3) Роль в бизнес-процессах

Изначально данные хранятся в обычных БД и уже оттуда поступают в DWH. Иными словами, Data Warehouse всегда содержит последние версии данных.



Data Warehouse: архитектура

Архитектура хранилища данных включает несколько уровней.

Верхний уровень — интерфейсный клиент, предоставляющий результаты с использованием инструментов формирования отчетов, поиска и анализа данных.

Средний уровень — аналитический механизм, который используется для доступа к данным и их анализа.

Нижний уровень архитектуры — сервер базы данных, отвечающий за загрузку и хранение данных.

Data Warehouse: хранение данных

Данные хранятся двумя разными способами:

- 1) данные, к которым нужен частый доступ, хранятся в очень быстром хранилище (например, на SSD-дисках);
- 2) данные с нечастым доступом хранятся в более дешевом хранилище объектов, например в Amazon S3.

Хранилище данных автоматически выполняет перенос часто используемых данных в «быстрое» хранилище для оптимизации скорости запросов.

Data Warehouse: сценарий использования



Загрузка данных в базу данных или озеро данных, предварительная обработка данных и перемещение нужных частей в хранилище данных, затем подготовка отчетов.

Data Warehouse: сценарий использования



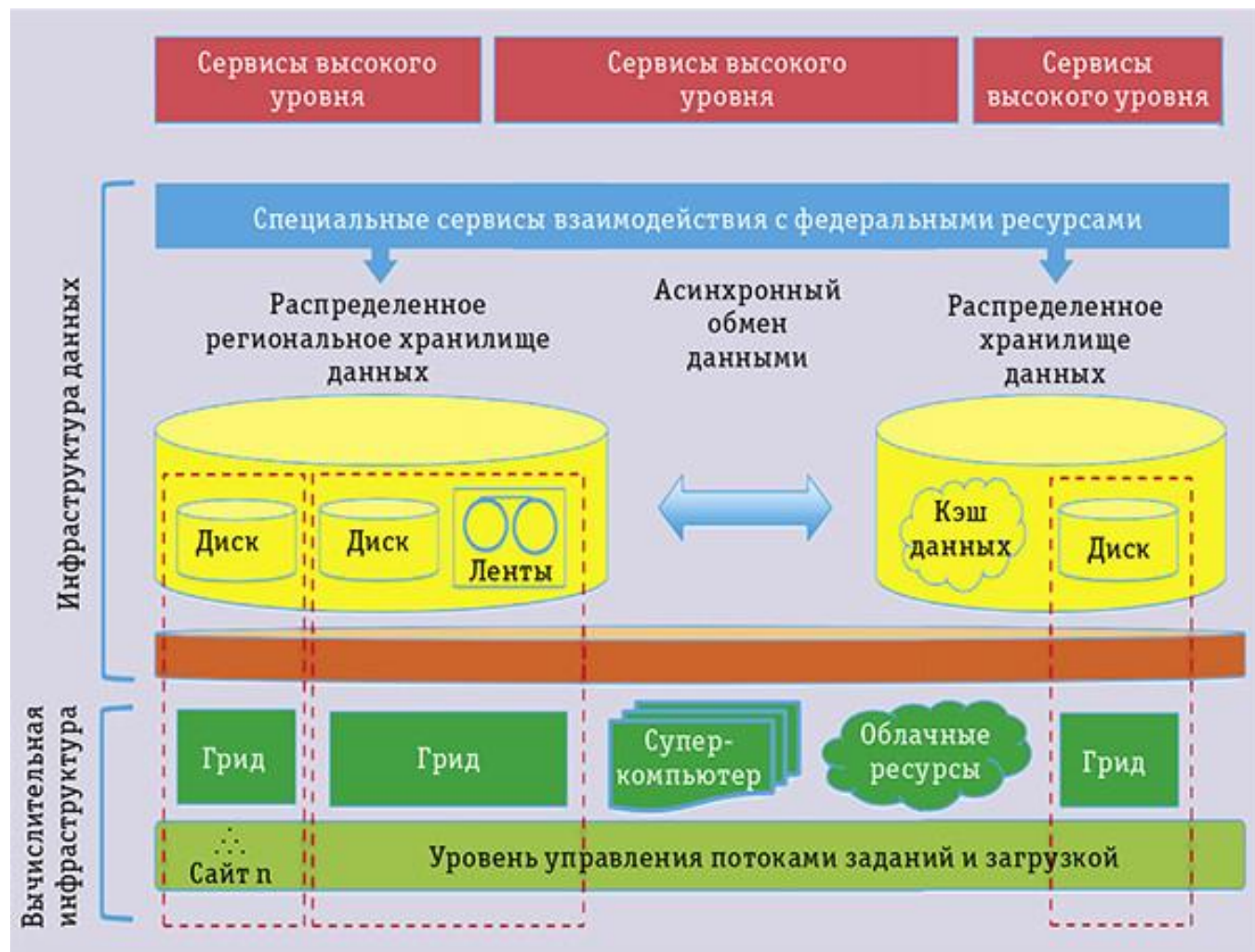
Загрузка данных в хранилище данных, анализ и предоставление общего доступа к данным для других сервисов аналитики и машинного обучения.

Data Lake: определение

Data Lake – это элемент инфраструктуры больших данных, хранилище сверхбольшого объёма неструктурированных данных, генерированных и/или собранных компанией. Данные в озерах хранятся, как правило, в **несистематизированном** виде.

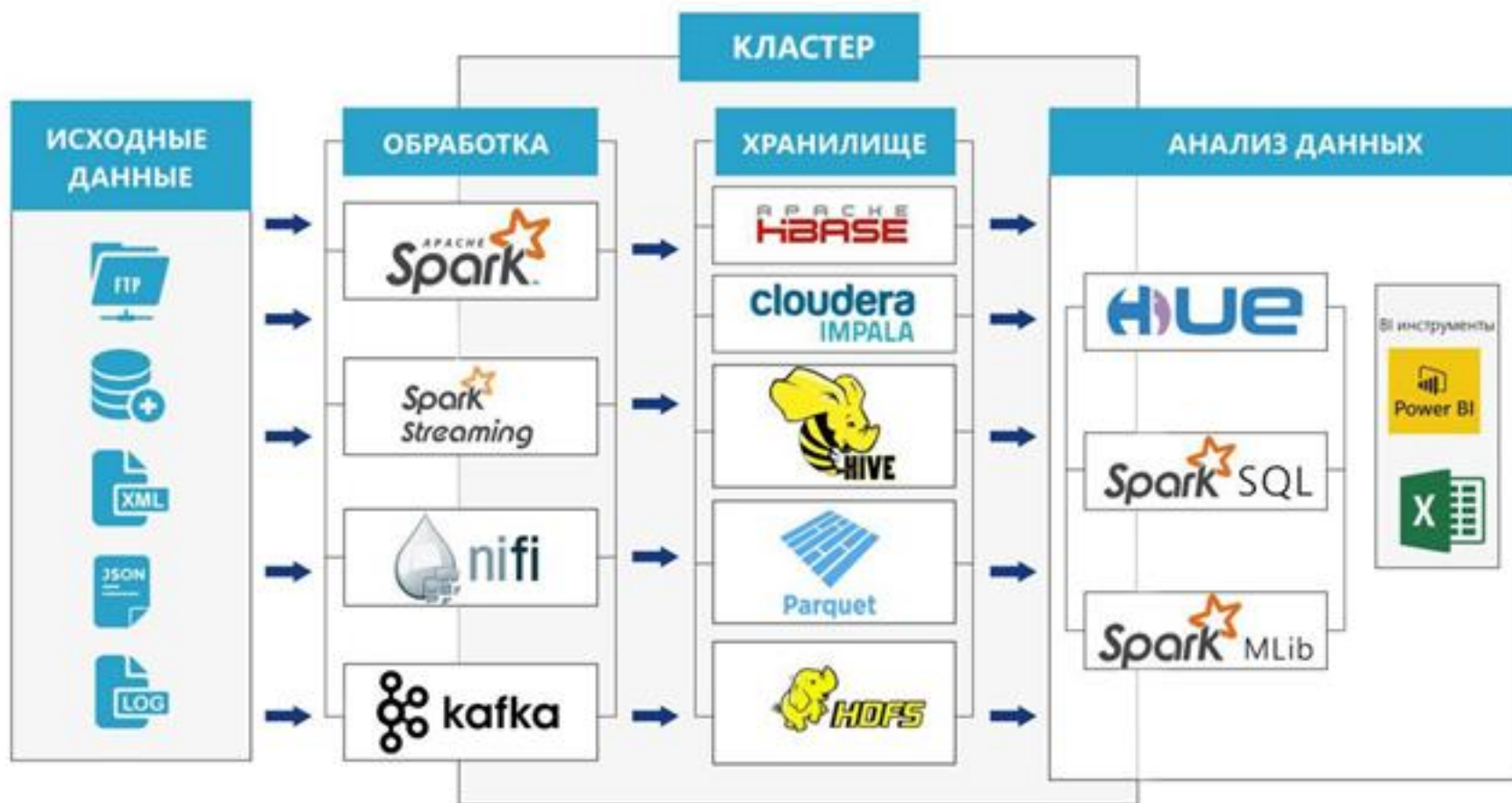
Идея озера данных заключается в наличии логического **определённого и единого хранилища всех данных** организации включая как сырые, необработанные данные, так и данные, прошедшие предварительную обработку. Хранимые данные используются для широкого спектра задач: формирование отчётов, визуализация, аналитика и машинное обучение.

Data Lake: архитектура



Вариант архитектуры научного озера данных

Data Lake: стек технологий



Вариант стека реализации компонент озера данных

Data Lake vs Data Warehouse

Озеро данных	Хранилище данных
Комплиментарно к хранилищу данных	Может использовать озеро данных в качестве источника данных
Составление схемы при чтении данных	Составление схемы при записи данных
Содержит структурированные, неструктурированные и полу-структурированные данные	Содержит только структурированные данные
Быстрая операция добавления данных	Операция добавления данных требует определённого времени
Разнообразные средства аналитики и BI	Встроенные средства BI
Данные представлены с низкой детализацией	Данные представлены в агрегированном виде
Слабо определённое соглашение об использовании	Строго определённое соглашение об использовании
Абсолютная гибкость встраиваемых решений	Относительная гибкость встраиваемых решений

Data Mart: определение

Data Mart сосредоточена на одной функциональной области организации и содержит подмножество данных, хранящихся в хранилище данных. Data Mart – это **сжатая версия хранилища данных**, предназначенная для использования конкретным отделом, подразделением или группой пользователей в организации. Например, маркетинг, продажи, HR или финансы. Это часто контролируется одним отделом в организации.

Data Mart обычно получает данные только из нескольких источников по сравнению с хранилищем данных. Витрины данных имеют небольшой размер и более гибкие по сравнению с хранилищем данных.

Data Mart: особенности

- Data Mart помогает увеличить время отклика пользователя за счет уменьшения объема данных
- Это обеспечивает легкий доступ к часто запрашиваемым данным.
- Стенд данных проще реализовать по сравнению с корпоративным хранилищем данных. В то же время стоимость внедрения Data Mart определено ниже по сравнению с полным хранилищем данных.
- По сравнению с хранилищем данных, data mart является гибким. В случае изменения модели, Data mart может быть построен быстрее из-за меньшего размера.

Data Mart: особенности

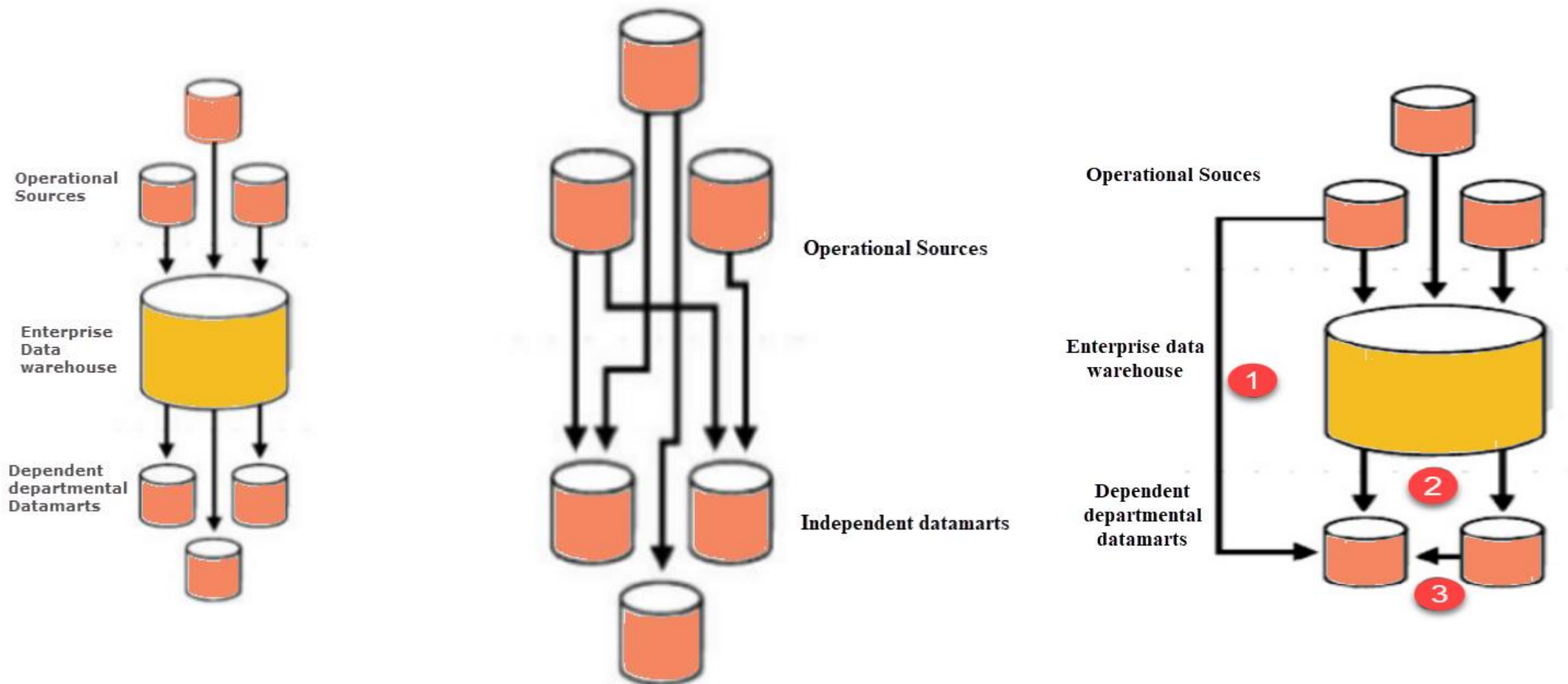
- Data mart определяется одним экспертом по предмету. Напротив, хранилище данных определяется междисциплинарным МСП из множества доменов. Следовательно, Data Mart более открыт для изменений по сравнению с Data warehouse.
- Данные разделены на части и предоставляют очень детальные права доступа.
- Данные могут быть сегментированы и храниться на разных аппаратных / программных платформах.

Data Mart: типы

Существует три основных типа витрин данных:

- **зависимый:** зависимые витрины данных создаются путем потребления данных непосредственно из операционных, внешних или обоих источников.
- **независимый:** независимые витрины данных создаются без использования центрального хранилища данных.
- **гибридные:** витрины данных этого типа могут получать данные из хранилищ данных или операционных систем.

Data Mart: архитектура типов





ClickHouse: определение

ClickHouse – столбцовая система управления базами данных (СУБД) для онлайн обработки аналитических запросов (OLAP).

<https://clickhouse.com/>

ClickHouse: структура данных

Строка	WatchID	JavaEnable	Title	GoodEvent	EventTime
#0	89354350662	1	Investor Relations	1	2016-05-18 05:19:20
#1	90329509958	0	Contact us	1	2016-05-18 08:10:20
#2	89953706054	1	Mission	1	2016-05-18 07:38:00
#N	...	...	...	...	...

Строковая СУБД

Строка:	#0	#1	#2	#N
WatchID:	89354350662	90329509958	89953706054	...
JavaEnable:	1	0	1	...
Title:	Investor Relations	Contact us	Mission	...
GoodEvent:	1	1	1	...
EventTime:	2016-05-18 05:19:20	2016-05-18 08:10:20	2016-05-18 07:38:00	...

Столбцовая СУБД 48



ClickHouse: особенности

- подавляющее большинство запросов – **на чтение**;
- данные обновляются достаточно большими пачками (> 1000 строк), а не по одной строке, или не обновляются вообще;
- данные добавляются в БД, но **не изменяются**;
- при чтении, вынимается достаточно большое количество строк из БД, но только небольшое подмножество столбцов;
- таблицы являются «широкими», то есть, содержат большое количество столбцов;
- запросы идут сравнительно редко (обычно не более сотни в секунду на сервер);



ClickHouse: особенности

- при выполнении простых запросов, допустимы задержки в районе 50 мс;
- значения в столбцах достаточно мелкие – числа и небольшие строки (пример - 60 байт на URL);
- требуется высокая пропускная способность при обработке одного запроса (до миллиардов строк в секунду на один сервер);
- **транзакции отсутствуют;**

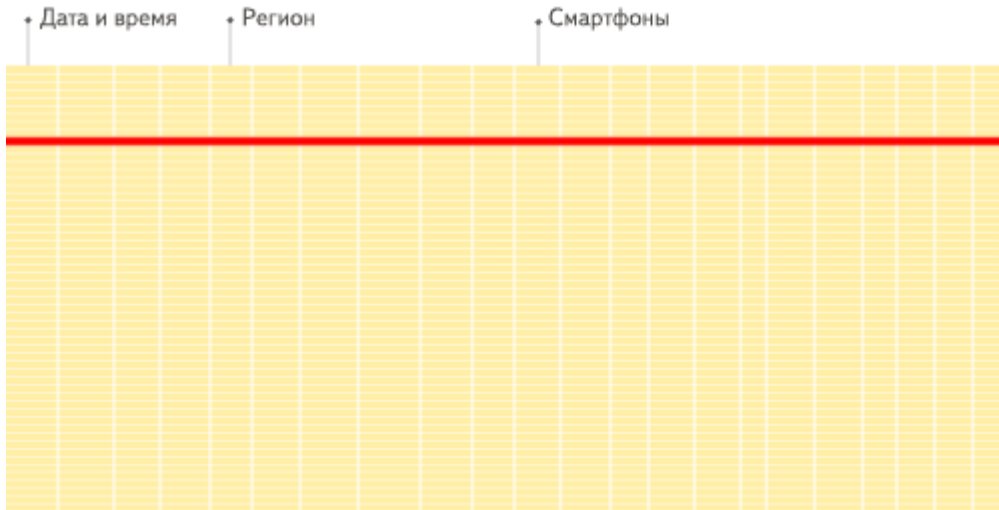


ClickHouse: особенности

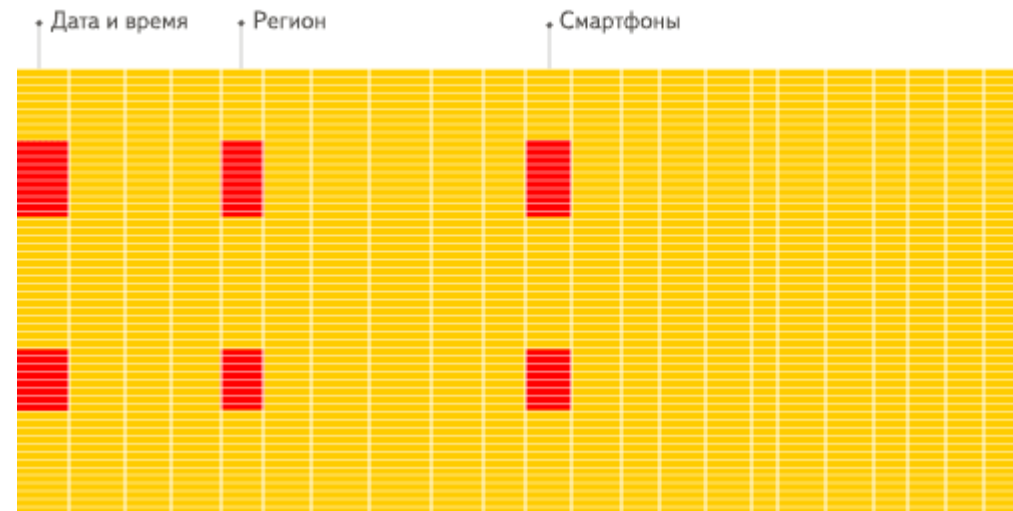
- низкие требования к консистентности данных;
- в запросе одна большая таблица, все таблицы кроме одной маленькие;
- результат выполнения запроса существенно меньше исходных данных – то есть, данные **фильтруются** или **агрегируются**; результат выполнения помещается в ОЗУ на одном сервере.



ClickHouse: скорость работы



Строковая СУБД



Столбцовая СУБД



ClickHouse: ВВОД-ВЫВОД

- Для выполнения аналитического запроса, требуется прочитать небольшое количество столбцов таблицы. В столбцовой БД для этого можно читать **только нужные данные**. Например, если вам требуется только 5 столбцов из 100, то следует рассчитывать на 20-кратное уменьшение ввода-вывода.
- Так как данные читаются пачками, то их проще сжимать. Данные, лежащие по столбцам также лучше сжимаются. За счёт этого, дополнительно уменьшается объём ввода-вывода.
- За счёт уменьшения ввода-вывода, больше данных влезает в системный кэш.



ClickHouse: вычисления

- Векторный движок. Все операции пишутся не для отдельных значений, а **для векторов**. То есть, вызывать операции надо достаточно редко, и издержки на диспетчеризацию становятся пренебрежимо маленькими. Код операции содержит в себе хорошо оптимизированный внутренний цикл.
- Кодогенерация. Для запроса генерируется код, в котором подставлены **все косвенные вызовы**.



HDFS: определение

HDFS (Hadoop Distributed File System) — распределенная **файловая система** Hadoop для хранения файлов больших размеров с возможностью потокового доступа к информации, поблочной распределённой по узлам вычислительного кластера, который может состоять из произвольного аппаратного обеспечения.

Hadoop Distributed File System, как и любая файловая система — это **иерархия каталогов** с вложенными в них подкаталогами и файлами.

https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html



HDFS: применение

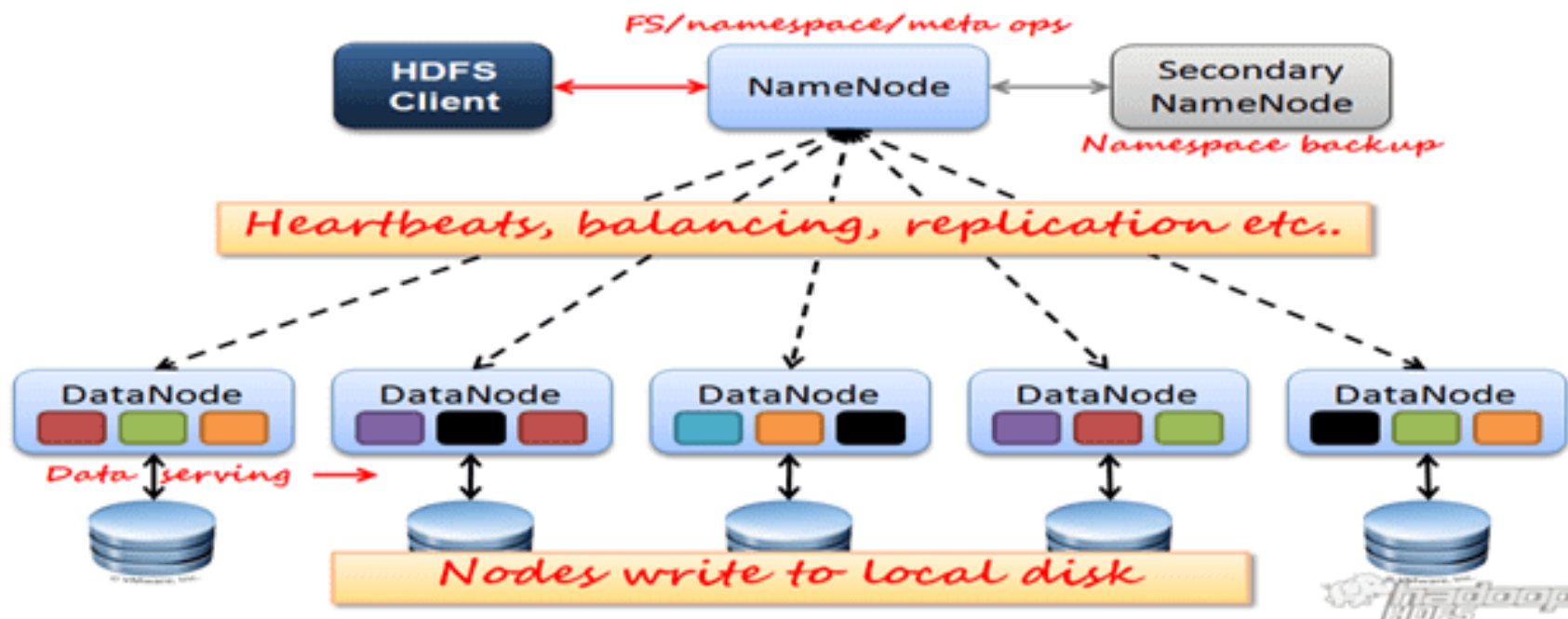
HDFS – неотъемлемая часть Hadoop, проекта верхнего уровня Apache Software Foundation, и основа инфраструктуры больших данных.

Однако, Hadoop поддерживает работу и с другими **распределёнными файловыми системами**, в частности, Amazon S3 и CloudStore.

Также некоторые дистрибутивы Hadoop, например, MapR, реализуют свою аналогичную распределенную файловую систему – MapR File System.

HDFS: применение

HDFS может использоваться не только для запуска **MapReduce**-заданий, но и как распределённая файловая система общего назначения, обеспечивая работу распределённых СУБД (HBase) и масштабируемых систем машинного обучения (Apache Mahout).



HDFS: архитектура

Управляющий узел, узел имен или сервер имен (NameNode) – отдельный, единственный в кластере, сервер с программным кодом для управления пространством имен файловой системы, хранящий **дерево файлов**, а также мета-данные файлов и каталогов.

NameNode – обязательный компонент кластера HDFS, который отвечает за открытие и закрытие файлов, создание и удаление каталогов, управление доступом со стороны внешних клиентов и соответствие между файлами и блоками, дублированными (реплицированными) на узлах данных. Сервер имен раскрывает для всех желающих **расположение блоков** данных на машинах кластера.

HDFS: архитектура

Secondary NameNode — **вторичный узел имен**, отдельный сервер, единственный в кластере, который **копирует образ HDFS** и лог транзакций операций с файловыми блоками во временную папку, применяет изменения, накопленные в логе транзакций к образу HDFS, а также записывает его на узел NameNode и очищает лог транзакций.

Secondary NameNode необходим для **быстрого ручного восстановления** NameNode в случае его выхода из строя.

HDFS: архитектура

Узел или **сервер данных** (DataNode, Node) – один из множества серверов кластера с программным кодом, отвечающим за файловые операции и работу с блоками данных.

DataNode – обязательный компонент кластера HDFS, который отвечает за **запись и чтение данных**, выполнение команд от узла NameNode по созданию, удалению и репликации блоков, а также периодическую **отправку сообщения о состоянии** (heartbeats) и обработку запросов на чтение и запись, поступающих от клиентов файловой системы HDFS. Стоит отметить, что данные проходят с остальных узлов кластера к клиенту мимо узла NameNode.

HDFS: архитектура

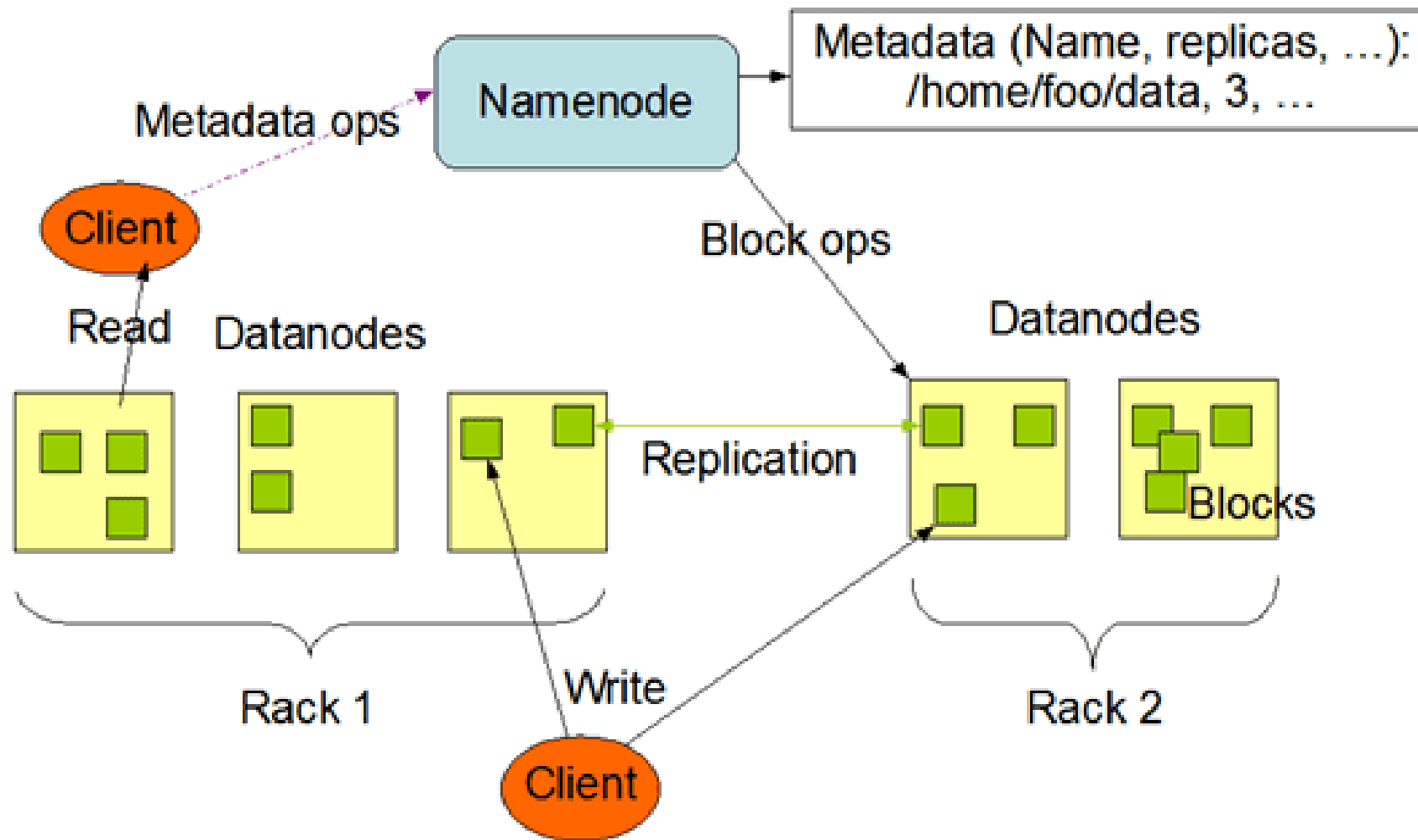
Клиент (client) – пользователь или приложение, взаимодействующий **через API** с распределенной файловой системой.

При наличии достаточных прав, клиенту разрешены следующие операции с файлами и каталогами: создание, удаление, чтение, запись, переименование и перемещение.

Создавая файл, клиент может **явно указать** размер блока файла (64 Мб) и количество создаваемых реплик (3).

Взаимодействие узлов имен, узлов данных и клиентов осуществляется по протоколам на основе **TCP/IP**.

HDFS: архитектура



HDFS: особенности

- **большой размер блока** по сравнению с другими файловыми системами ($>64\text{MB}$), поскольку HDFS предназначена для хранения **большого количества** огромных ($>10\text{GB}$) файлов;
- ориентация на недорогие и, поэтому **не самые надежные** сервера – отказоустойчивость всего кластера обеспечивается за счет репликации данных;
- зеркалирование и **репликация** осуществляются на уровне кластера, а не на уровне узлов данных;
- репликация происходит в **асинхронном режиме** – информация распределяется по нескольким серверам прямо во время загрузки, поэтому выход из строя отдельных узлов данных не повлечет за собой полную пропажу данных;

HDFS: особенности

- HDFS оптимизирована для **потоковых считываний файлов**, поэтому применять ее для нерегулярных и произвольных считываний нецелесообразно;
- клиенты могут считывать и писать файлы HDFS напрямую через программный интерфейс **Java**;
- файлы пишутся **однократно**, что исключает внесение в них любых произвольных изменений;
- принцип **WORM** (Write-once and read-many, один раз записать – много раз прочитать) полностью освобождает систему от блокировок типа «запись-чтение». Запись в файл в одно время доступен только одному процессу, что исключает конфликты множественной записи.

HDFS: особенности

- HDFS оптимизирована под **потокковую передачу данных**;
- сжатие данных и рациональное использование дискового пространства **позволило снизить нагрузку** на каналы передачи данных, которые чаще всего являются узким местом в распределенных средах;
- **самодиагностика** — каждый узел данных через определенные интервалы времени отправляет диагностические сообщения узлу имен, который записывает логи операций над файлами в специальный журнал;
- все метаданные сервера имен хранятся в **оперативной памяти**.

HDFS: недостатки

- сервер имен является центральной точкой всего кластера и его отказ повлечет **сбой системы целиком**;
- отсутствие **полноценной репликации** Secondary NameNode;
- отсутствие возможности дописывать или оставить открытым для записи файлы в HDFS, за счет чего в классическом дистрибутиве Apache Hadoop **невозможно обновлять блоки** уже записанных данных;
- **отсутствие поддержки реляционных моделей данных**;

HDFS: недостатки

Отсутствие инструментов для поддержки ссылочной целостности данных, что **не гарантирует** идентичность реплик.

HDFS перекладывает проверку целостности данных на **клиентов**. При создании файла клиент рассчитывает контрольные суммы каждые 512 байт, которые в последующем сохраняются на сервере имен.

При считывании файла клиент обращается к данным и контрольным суммам. В случае их несоответствия происходит обращение к другой реплике.



Greenplum: определение

Greenplum – массивно-параллельная реляционная СУБД для хранилищ данных с гибкой горизонтальной масштабируемостью и столбцовым хранением данных на основе PostgreSQL.

<https://greenplum.org/>



Greenplum: архитектура

Greenplum представляет собой несколько взаимосвязанных экземпляров базы данных PostgreSQL, объединенных в кластер по принципу **массивно-параллельной архитектуры** (Massive Parallel Processing, MPP) **без разделения ресурсов** (Shared Nothing).

При этом каждый узел кластера, взаимодействующий с другими для выполнения вычислительных операций, имеет **собственную память**, операционную систему и жесткие диски.

Для повышения надежности к типовой топологии master-slave добавлен резервный главный сервер.



Greenplum: архитектура

Мастер-сервер (Master host), где развернут главный инстанс PostgreSQL (Master instance). Это точка входа в Greenplum, куда подключаются клиенты, отправляя SQL-запросы.

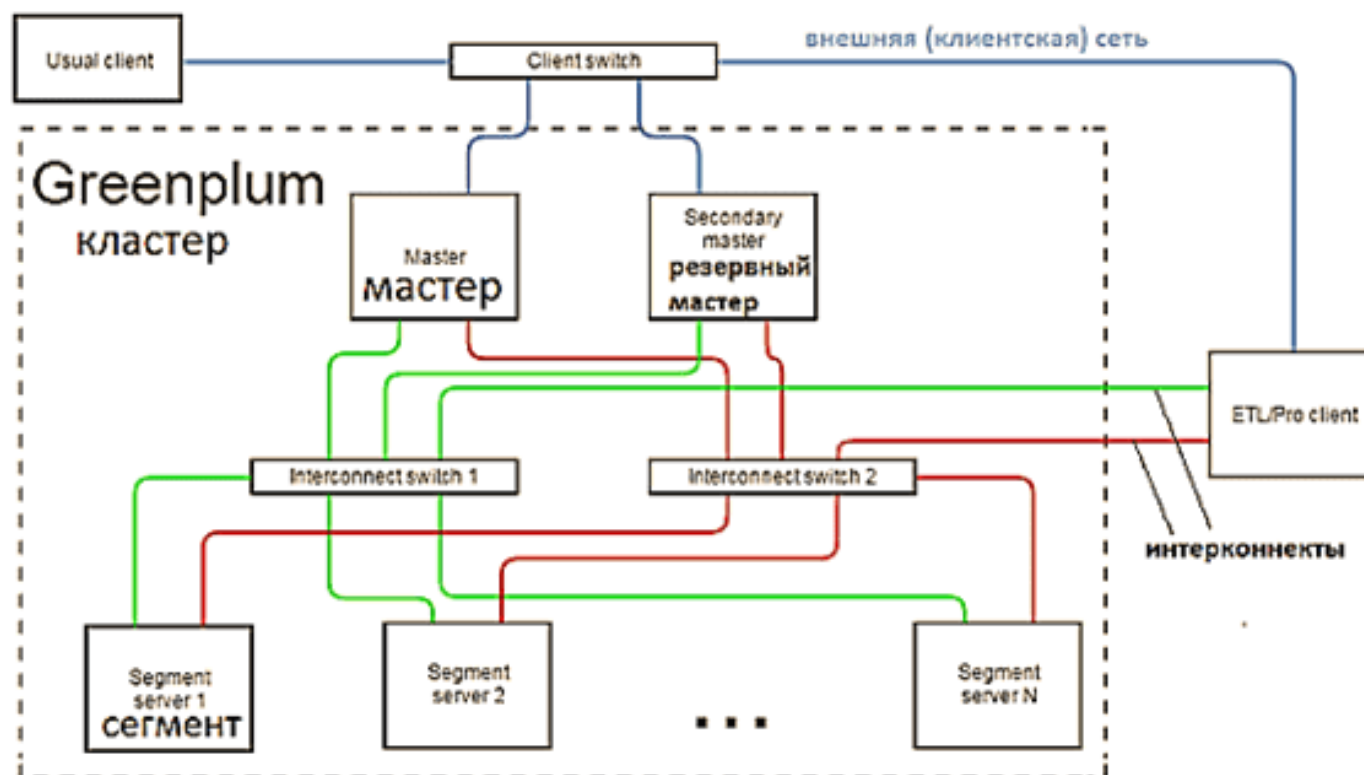
Мастер координирует свою работу с сегментами – другими экземплярами базы данных PostgreSQL.

Мастер распределяет нагрузку между сегментами, но сам **не содержит никаких пользовательских данных** – они хранятся только на сегментах.



Greenplum: архитектура

Резервный мастер (Secondary master instance) — инстанс PostgreSQL, включаемый вручную при отказе основного мастера.





Greenplum: архитектура

Сервер-сегмент (Segment host), где хранятся и обрабатываются данные. На одном хост-сегменте содержится 2-8 сегментов Greenplum – независимых экземпляров PostgreSQL с частью данных. Сегменты Гринплам бывают основные (primary) и зеркальные (mirror).

Primary-сегмент обрабатывает локальные данные и отдает результаты мастеру. **Каждому** primary-сегменту соответствует **свое** зеркало (Mirror segment instance), которое **автоматически** включается в работу при отказе primary.

Интерконнект (interconnect) – быстрое обособленное сетевое соединение для связи между отдельными экземплярами PostgreSQL.



Greenplum: взаимодействие

- пользователь подключается к базе данных с **помощью клиентских программ**: psql или через API-интерфейсы типа JDBC и ODBC;
- мастер **аутентифицирует** клиентские соединения и обрабатывает входящие SQL-запросы;
- **каждый** сегмент для обработки запроса создает соответствующие **процессы**;
- после выполнения вычислений над локальными данными сегмент **возвращает результаты** мастеру;
- мастер координирует результаты от сегментов и представляет **конечный итог** клиентской программе.

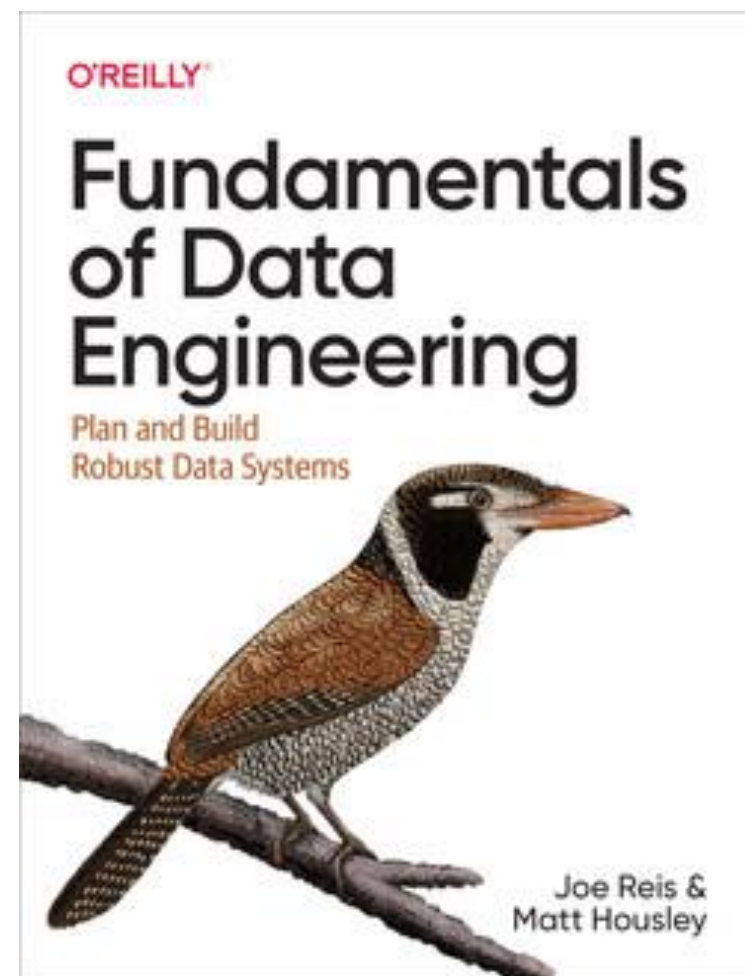


Greenplum: применение

- системы предиктивной аналитики и регулярной отчетности по большим объемам данных;
- построение озер и корпоративных хранилищ данных;
- разработка аналитических моделей по множеству разнообразных данных, например, для прогнозирования оттока клиентов (Churn Rate).

Основы инжиниринга данных

- Обзор всей области инженерии данных;
- Решение актуальных проблем, используя рекомендованные практики;
- Обоснование выбор технологий, архитектур и процессов работы с данными;
- Жизненный цикл отказоустойчивой архитектуры инженерии данных;
- Обеспечение качества и безопасности данных.



Вопросы

?